Lista 6 - Redes de Computadores I

Nome: Thiago Henrique Gomes Feliciano

1) Início de conexão TCP (Three-Way Handshake):

```
Problems 39   Output   Debug Console   Terminal   Ports

PS C:\Users\thigo\Downloads\tcp_trabalho> javac Servidor.java Cliente.java
PS C:\Users\thigo\Downloads\tcp_trabalho> java Servidor
Servidor pronto
```

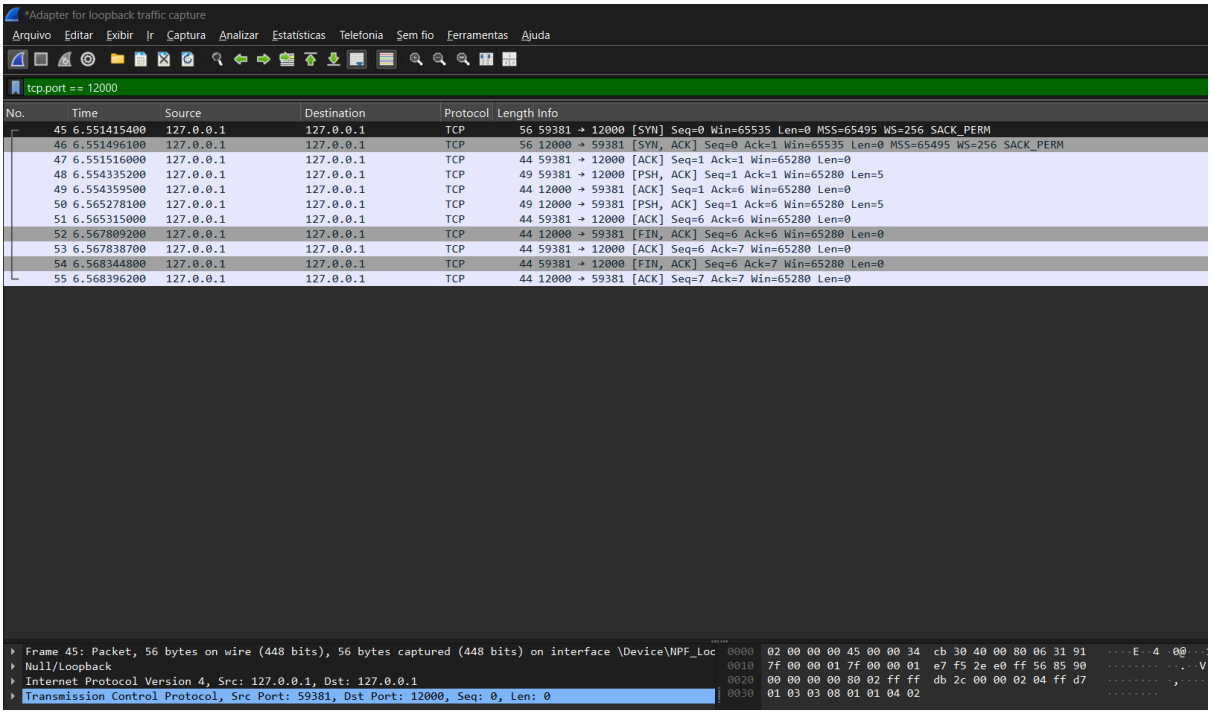
```
Problems 39   Output   Debug Console   Terminal   Ports

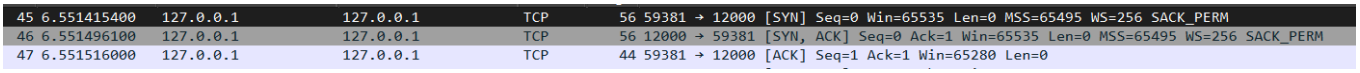
PS C:\Users\thigo\Downloads\tcp_trabalho> java Cliente
Frase em minusculas: hello
Resposta: HELLO
PS C:\Users\thigo\Downloads\tcp_trabalho>
```

Identifique os três segmentos TCP utilizados para estabelecer a conexão entre o cliente e o servidor.

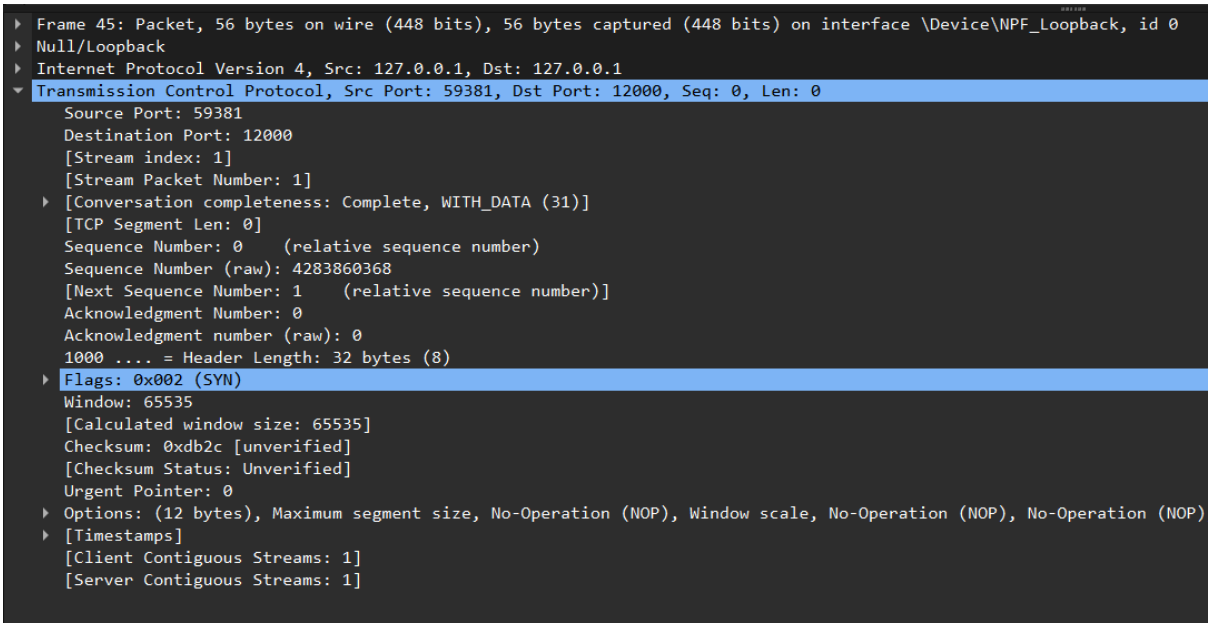
a) Apresente os segmentos que contêm as combinações de flags SYN, SYN/ACK e ACK.

R:

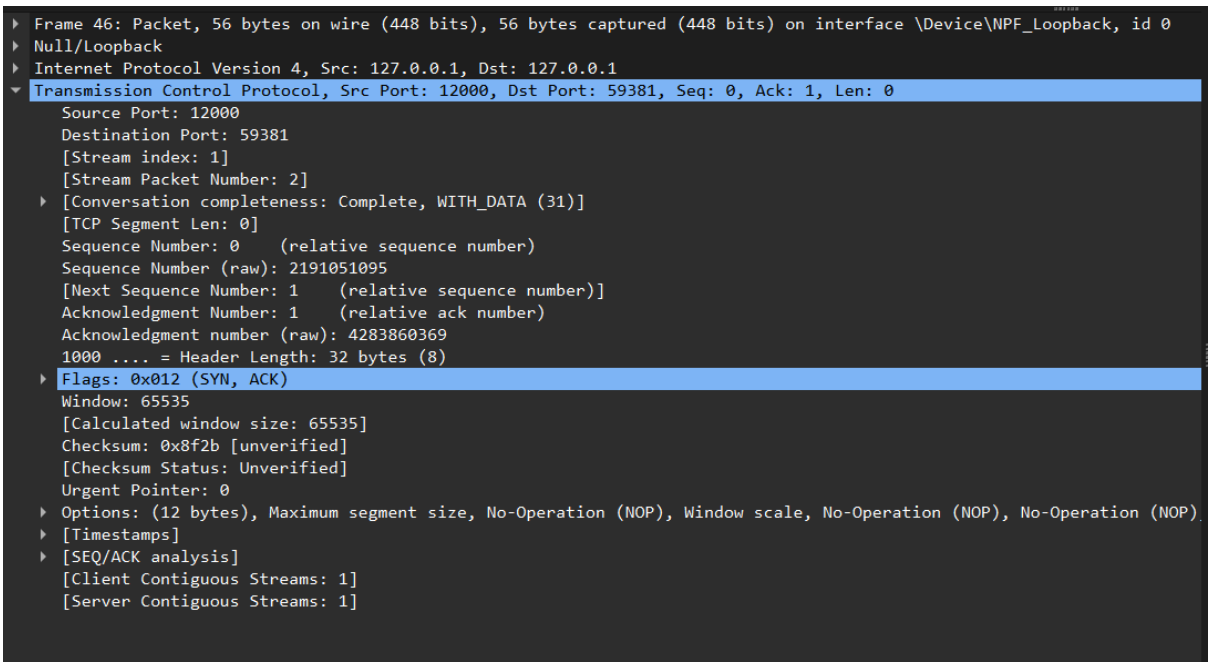




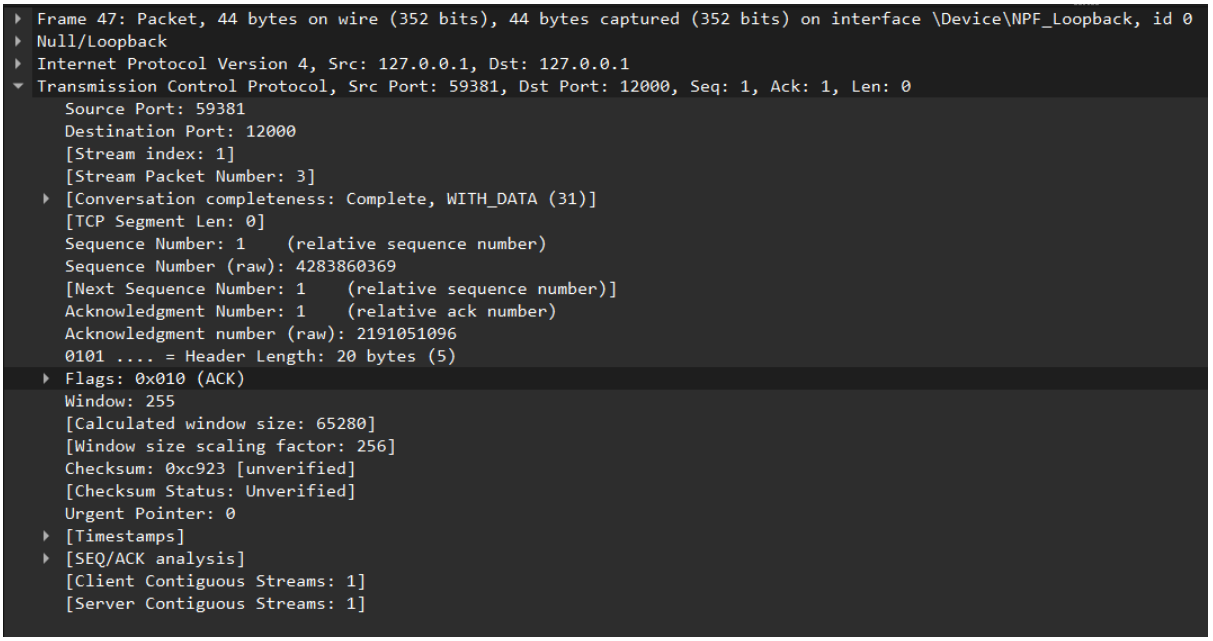
45	6.551415400	127.0.0.1	127.0.0.1	TCP	56 59381 → 12000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
46	6.551496100	127.0.0.1	127.0.0.1	TCP	56 12000 → 59381 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
47	6.551516000	127.0.0.1	127.0.0.1	TCP	44 59381 → 12000 [ACK] Seq=1 Ack=1 Win=65280 Len=0



```
▸ Frame 45: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on interface \Device\NPF_Loopback, id 0
▸ Null/Loopback
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▾ Transmission Control Protocol, Src Port: 59381, Dst Port: 12000, Seq: 0, Len: 0
    Source Port: 59381
    Destination Port: 12000
    [Stream index: 1]
    [Stream Packet Number: 1]
  ▸ [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 0]
    Sequence Number: 0    (relative sequence number)
    Sequence Number (raw): 4283860368
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 0
    Acknowledgment number (raw): 0
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x002 (SYN)
    Window: 65535
    [Calculated window size: 65535]
    Checksum: 0xdb2c [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP)
  ▸ [Timestamps]
    [Client Contiguous Streams: 1]
    [Server Contiguous Streams: 1]
```



```
▸ Frame 46: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on interface \Device\NPF_Loopback, id 0
▸ Null/Loopback
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▾ Transmission Control Protocol, Src Port: 12000, Dst Port: 59381, Seq: 0, Ack: 1, Len: 0
    Source Port: 12000
    Destination Port: 59381
    [Stream index: 1]
    [Stream Packet Number: 2]
  ▸ [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 0]
    Sequence Number: 0    (relative sequence number)
    Sequence Number (raw): 2191051095
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 4283860369
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x012 (SYN, ACK)
    Window: 65535
    [Calculated window size: 65535]
    Checksum: 0x8f2b [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP)
  ▸ [Timestamps]
  ▸ [SEQ/ACK analysis]
    [Client Contiguous Streams: 1]
    [Server Contiguous Streams: 1]
```



b) Identifique os campos Sequence Number e Acknowledgment Number dos três segmentos.

R:

Nº	Flags	Seq (relativo)	Ack (relativo)
45	SYN	0	não usado (flag ACK desligada)
46	SYN, ACK	0	1
47	ACK	1	1

O Seq do pacote 45 é o número de sequência inicial (ISN) do cliente. O Seq do pacote 46 é o ISN do servidor. O Ack=1 no pacote 46 significa que o servidor recebeu o SYN do cliente (que ocupa o número 0) e espera o próximo número, 1. No pacote 47, o Ack=1 faz o mesmo para o SYN do servidor.

O SYN consome um número de sequência, mesmo sem carregar dados. Por isso o cliente passa de Seq=0 para Seq=1 no terceiro segmento.

c) Explique a função de cada segmento e como o cliente e o servidor confirmam o estabelecimento da conexão.

R:
SYN (45): o cliente pede a abertura da conexão e informar seu número de sequência inicial. Também anuncia opções: MSS=65495 (maior segmento que aceita, típico de loopback), WS=256 (fator de escala da janela) e SACK_PERM (aceita confirmações seletivas). O tamanho do quadro é 56 bytes, contra 44 bytes de um segmento sem opções, e a diferença (12 bytes) são essas opções.

SYN/ACK (46): o servidor aceita o pedido. A flag ACK confirma o SYN do cliente. A flag SYN abre o sentido contrário, com o ISN do servidor e suas próprias opções. Os dois sentidos da conexão (full-duplex) têm numeração independente, e esse segmento sincroniza o do servidor.

ACK (47): o cliente confirma o SYN do servidor, sem carregar dados (Len=0).

Como cada lado confirma o estabelecimento: o cliente considera a conexão aberta ao receber o SYN/ACK. O servidor só a considera aberta ao receber o ACK final. Antes disso o servidor está em SYN_RCVD, e esse ACK prova que o cliente recebeu o seu SYN e que o caminho funciona nos dois sentidos.

O handshake durou cerca de 0,1 ms (de 6,551415 s a 6,551516 s), por ser tráfego local.

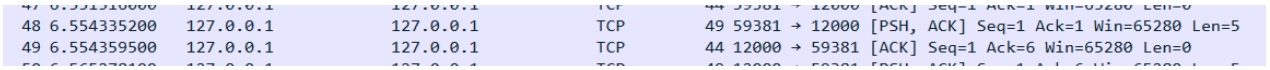
---

2) Transferência confiável de dados:

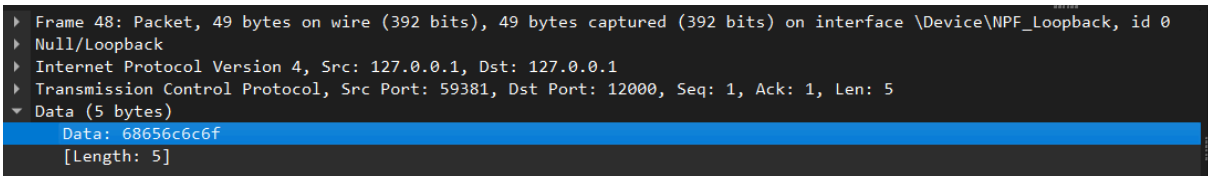
Identifique os segmentos TCP relacionados ao envio da frase do cliente para o servidor e à confirmação de seu recebimento.

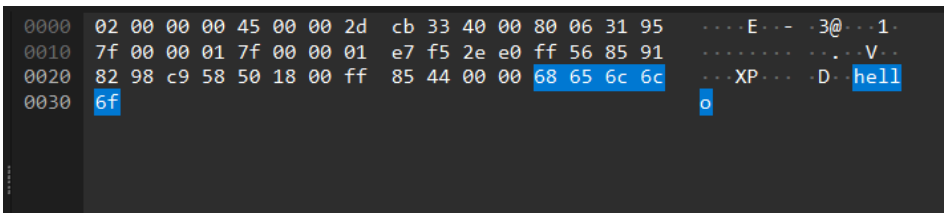
a) Apresente o segmento que transporta a frase enviada pelo cliente e o segmento que confirma seu recebimento.

R:



Transporta a frase enviada pelo cliente:





```
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▾ Transmission Control Protocol, Src Port: 59381, Dst Port: 12000, Seq: 1, Ack: 1, Len: 5
    Source Port: 59381
    Destination Port: 12000
    [Stream index: 1]
    [Stream Packet Number: 4]
  ▸ [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 5]
    Sequence Number: 1    (relative sequence number)
    Sequence Number (raw): 4283860369
    [Next Sequence Number: 6    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 2191051096
    0101 .... = Header Length: 20 bytes (5)
  ▸ Flags: 0x018 (PSH, ACK)
    Window: 255
    [Calculated window size: 65280]
    [Window size scaling factor: 256]
    Checksum: 0x8544 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ [Timestamps]
  ▸ [SEQ/ACK analysis]
    [Client Contiguous Streams: 1]
    [Server Contiguous Streams: 1]
    TCP payload (5 bytes)
▾ Data (5 bytes)
    Data: 68656c6c6f
    [Length: 5]
```

Segmento que confirma seu recebimento:

```
▸ Frame 49: Packet, 44 bytes on wire (352 bits), 44 bytes captured (352 bits) on interface \Device\NPF_Loopback, id 0
▸ Null/Loopback
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▸ Transmission Control Protocol, Src Port: 12000, Dst Port: 59381, Seq: 1, Ack: 6, Len: 0
```

```
▾ Transmission Control Protocol, Src Port: 12000, Dst Port: 59381, Seq: 1, Ack: 6, Len: 0
    Source Port: 12000
    Destination Port: 59381
    [Stream index: 1]
    [Stream Packet Number: 5]
  ▸ [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 0]
    Sequence Number: 1    (relative sequence number)
    Sequence Number (raw): 2191051096
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 6    (relative ack number)
    Acknowledgment number (raw): 4283860374
    0101 .... = Header Length: 20 bytes (5)
  ▸ Flags: 0x010 (ACK)
    Window: 255
    [Calculated window size: 65280]
    [Window size scaling factor: 256]
    Checksum: 0xc91e [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ [Timestamps]
  ▸ [SEQ/ACK analysis]
    [Client Contiguous Streams: 1]
    [Server Contiguous Streams: 1]
```

b) Identifique os campos Sequence Number, Acknowledgment Number e a flag ACK.

R:

Pacote 48 (dados):

- Sequence Number: 1
- Next Sequence Number: 6
- TCP Segment Len: 5
- Acknowledgment Number: 1
- Flags: 0x018 (PSH, ACK)

Pacote 49 (confirmação):

- Sequence Number: 1
- Acknowledgment Number: 6
- TCP Segment Len: 0
- Flags: 0x010 (ACK)

A flag ACK está ligada nos dois segmentos. O Wireshark mostra Acknowledgment: Set dentro de *Flags*. No pacote 48 o ACK confirma o SYN do servidor (Ack=1), e no 49 confirma os dados do cliente.

c) Explique como o número de sequência e o número de reconhecimento (ACK) permitem acompanhar a entrega dos dados. Relacione o valor do ACK à quantidade de bytes recebidos.

R: O TCP numera cada byte enviado. O Sequence Number é o número do primeiro byte do segmento, e o Acknowledgment Number é o número do próximo byte que o receptor espera receber.

- O pacote 48 leva 5 bytes, com números de sequência 1, 2, 3, 4 e 5. Por isso o próximo número do cliente é 6 (Next Sequence Number).
- O servidor responde com Ack=6, que significa “recebi todos os bytes até o 5 e espero o 6”.
- A relação é direta: Ack = Seq + Len = 1 + 5 = 6. Dessa forma, o valor do ACK menos o número do primeiro byte de dados (1) dá a quantidade de bytes recebidos em ordem: 6 − 1 = 5 bytes.

Esse tipo de confirmação é cumulativa: um Ack=N confirma todos os bytes anteriores a N de uma vez. Se um segmento se perdesse, o receptor continuaria enviando o mesmo Ack, e o emissor perceberia que faltam dados. Também é assim que o emissor sabe até onde pode descartar dados do seu buffer.

Neste caso, o ACK chegou 24 µs depois do envio (de 6,554335 s a 6,554359 s), o que indica uma rede local sem atraso.

d) Identifique a resposta do servidor contendo a frase convertida para letras maiúsculas e explique como o cliente confirma seu recebimento.

R:

```
50 6.565278100   127.0.0.1   127.0.0.1   TCP   49 12000 → 59381 [PSH, ACK] Seq=1 Ack=6 Win=65280 Len=5
51 6.565315000   127.0.0.1   127.0.0.1   TCP   44 59381 → 12000 [ACK] Seq=6 Ack=6 Win=65280 Len=0
```

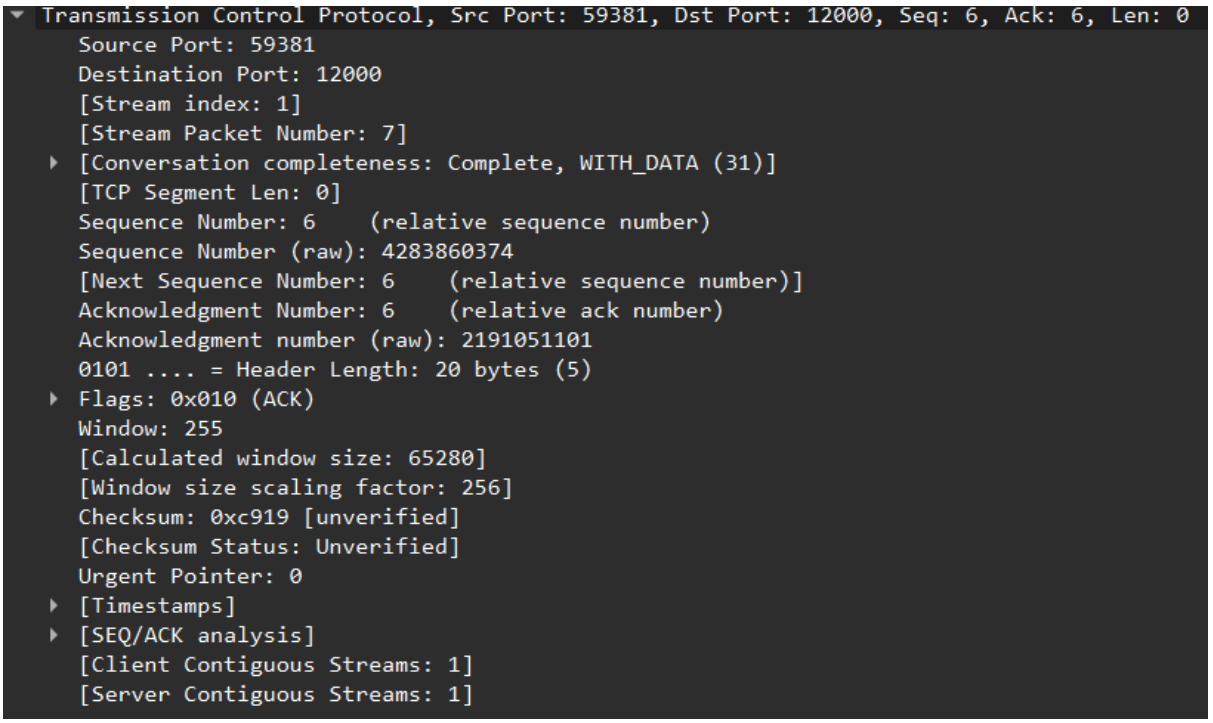
Resposta do servidor contendo a frase convertida para letras maiúsculas:

```
▸ Frame 50: Packet, 49 bytes on wire (392 bits), 49 bytes captured (392 bits) on interface \Device\NPF_Loopback, id 0
▸ Null/Loopback
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▸ Transmission Control Protocol, Src Port: 12000, Dst Port: 59381, Seq: 1, Ack: 6, Len: 5
▾ Data (5 bytes)
    Data: 48454c4c4f
    [Length: 5]
```

```
▾ Transmission Control Protocol, Src Port: 12000, Dst Port: 59381, Seq: 1, Ack: 6, Len: 5
    Source Port: 12000
    Destination Port: 59381
    [Stream index: 1]
    [Stream Packet Number: 6]
  ▸ [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 5]
    Sequence Number: 1    (relative sequence number)
    Sequence Number (raw): 2191051096
    [Next Sequence Number: 6    (relative sequence number)]
    Acknowledgment Number: 6    (relative ack number)
    Acknowledgment number (raw): 4283860374
    0101 .... = Header Length: 20 bytes (5)
  ▸ Flags: 0x018 (PSH, ACK)
    Window: 255
    [Calculated window size: 65280]
    [Window size scaling factor: 256]
    Checksum: 0xe57f [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ [Timestamps]
  ▸ [SEQ/ACK analysis]
    [Client Contiguous Streams: 1]
    [Server Contiguous Streams: 1]
    TCP payload (5 bytes)
```

Confirma a resposta:

```
▸ Frame 51: Packet, 44 bytes on wire (352 bits), 44 bytes captured (352 bits) on interface \Device\NPF_Loopback, id 0
▸ Null/Loopback
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▸ Transmission Control Protocol, Src Port: 59381, Dst Port: 12000, Seq: 6, Ack: 6, Len: 0
```



No pacote 50, o servidor envia 5 bytes (Seq=1, os bytes 1 a 5 do sentido servidor para cliente). O Ack=6 repete a confirmação dos dados do cliente, e a confirmação viaja junto com os dados (piggybacking). No campo Data aparece a frase em maiúsculas.

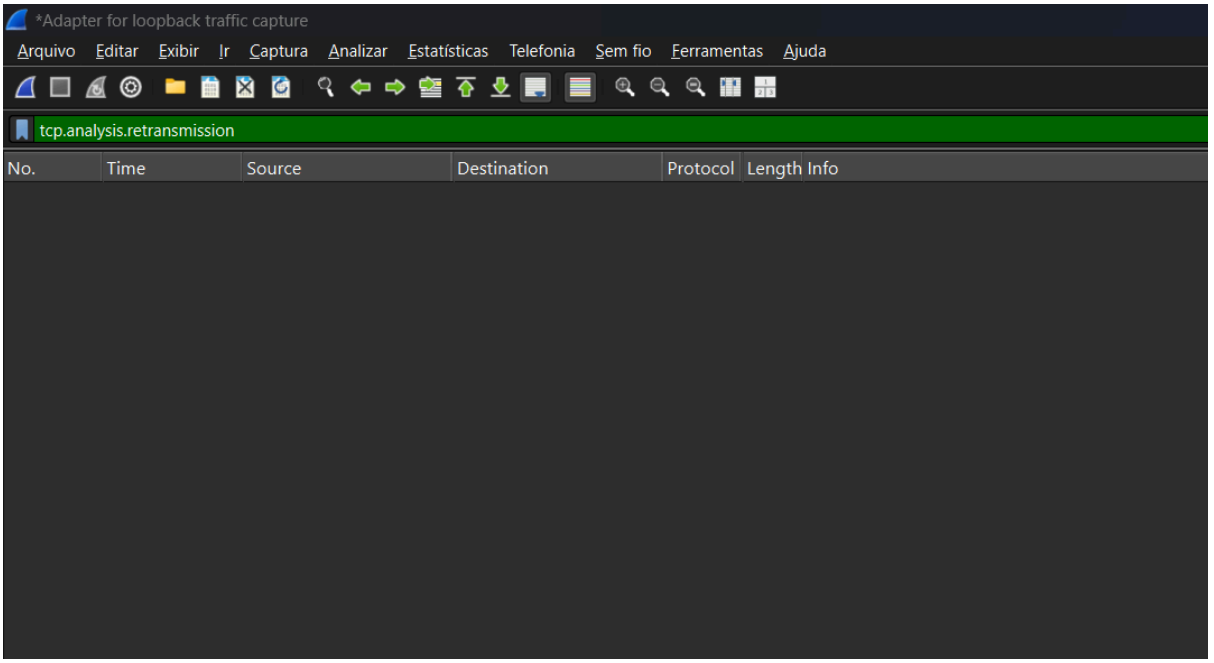
No pacote 51, o cliente envia Ack=6: “recebi os bytes 1 a 5 do servidor e espero o 6” (Ack = Seq do pacote 50 + Len = 1 + 5). O Seq=6 do cliente é o próximo byte que ele enviaria, o que bate com o Next Sequence Number do pacote 48. Como Len=0, o segmento serve só para confirmar.

O servidor levou cerca de 11 ms para processar a frase (de 6,554359 s a 6,565278 s), e o cliente confirmou 37 µs depois do recebimento

e) Verifique se ocorreram retransmissões durante a comunicação. Caso tenham ocorrido, identifique-as e explique sua função. Caso contrário, explique em que situações o TCP realiza retransmissões.

R: Não ocorreram retransmissões nesta captura. Os indícios são:

- Cada segmento de dados (48 e 50) foi confirmado em microssegundos.
- Nenhum número de sequência se repetiu com dados.
- O Wireshark não marcou nenhum pacote com [TCP Retransmission], [TCP Fast Retransmission] nem [TCP Dup ACK].



Em que situações o TCP retransmite:

1. Estouro do temporizador (timeout): o emissor inicia um temporizador (RTO) ao enviar um segmento. Se o ACK não chegar a tempo, ele reenvia o segmento, o que cobre perda do segmento ou do próprio ACK. Em geral o RTO é dobrado a cada nova tentativa.
2. Retransmissão rápida (fast retransmit): quando o emissor recebe três ACKs duplicados (mesmo Ack repetido), ele conclui que um segmento foi perdido e o reenvia sem esperar o timeout. O receptor gera esses ACKs duplicados ao receber segmentos fora de ordem.
3. Confirmações seletivas (SACK): como o SACK foi negociado no handshake (SACK_PERM), o receptor pode informar quais blocos chegaram depois de uma lacuna, e o emissor reenvia só o que falta.

A função da retransmissão é garantir a entrega completa e em ordem mesmo sobre o IP, que não garante nada. Em loopback não há perda de pacotes, e por isso nenhum desses mecanismos foi acionado.

---

3) Término de conexão TCP:

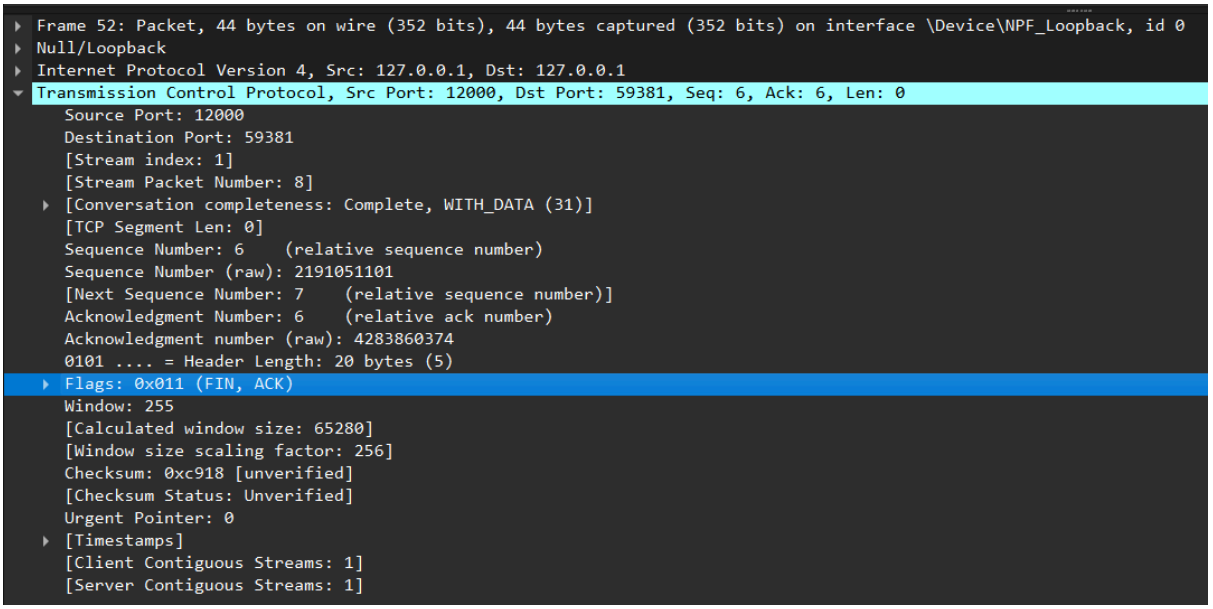
Identifique os segmentos TCP utilizados para encerrar normalmente a conexão entre o cliente e o servidor.

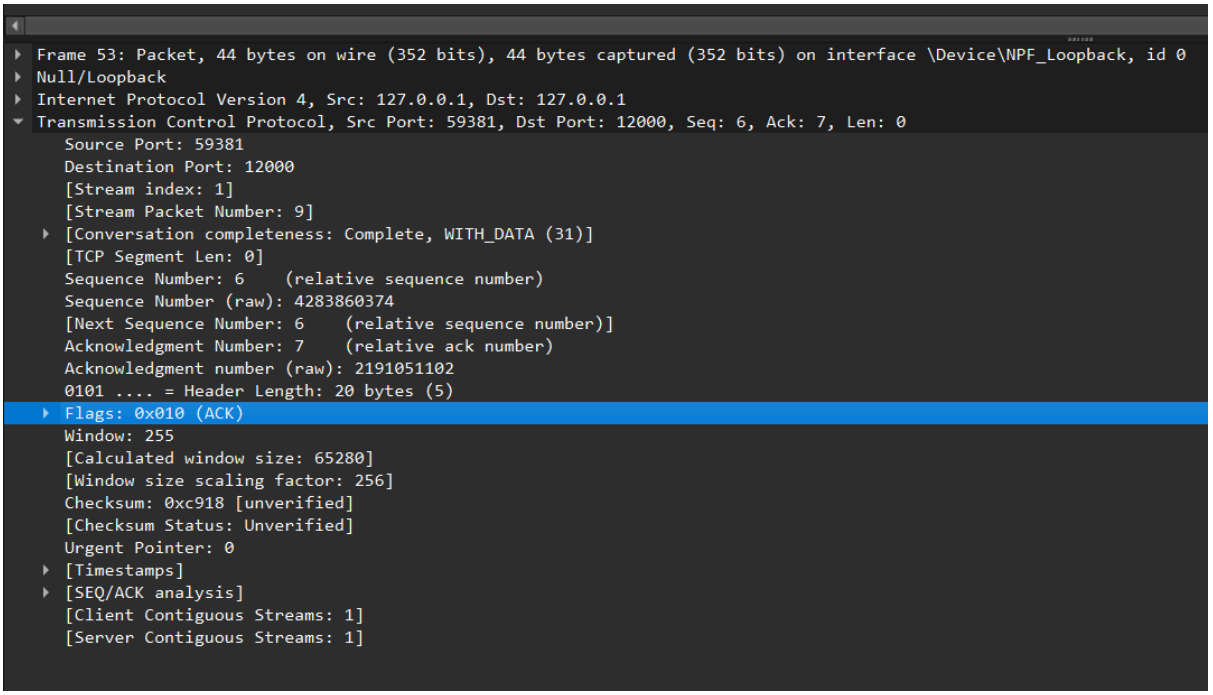
a) Apresente os segmentos que utilizam as flags FIN e ACK durante o encerramento.

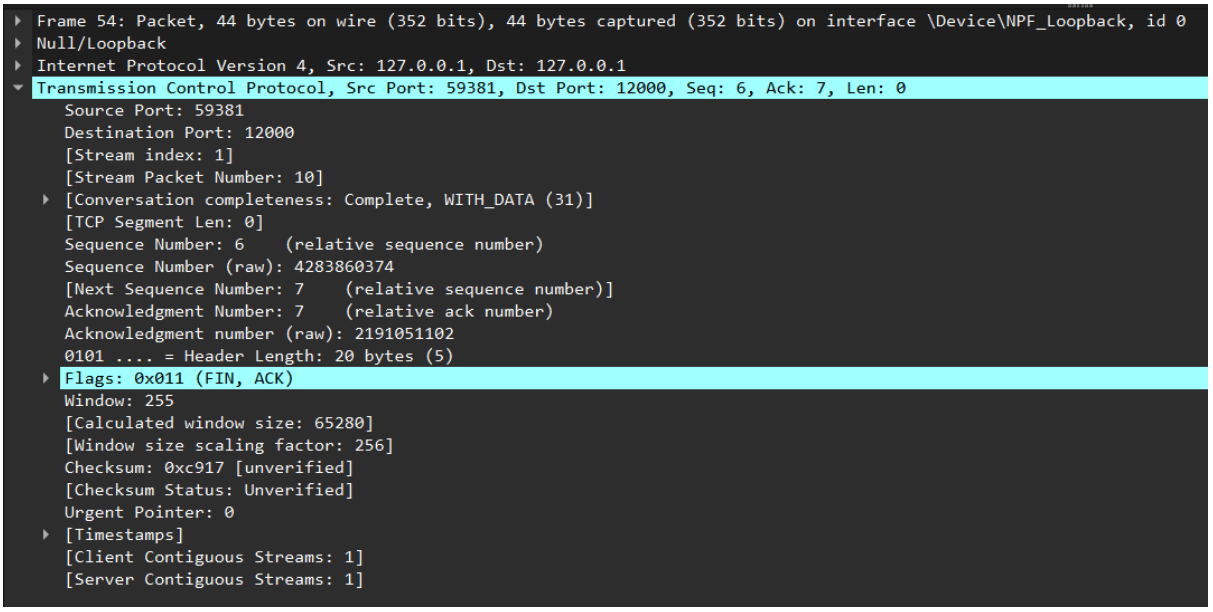
R:

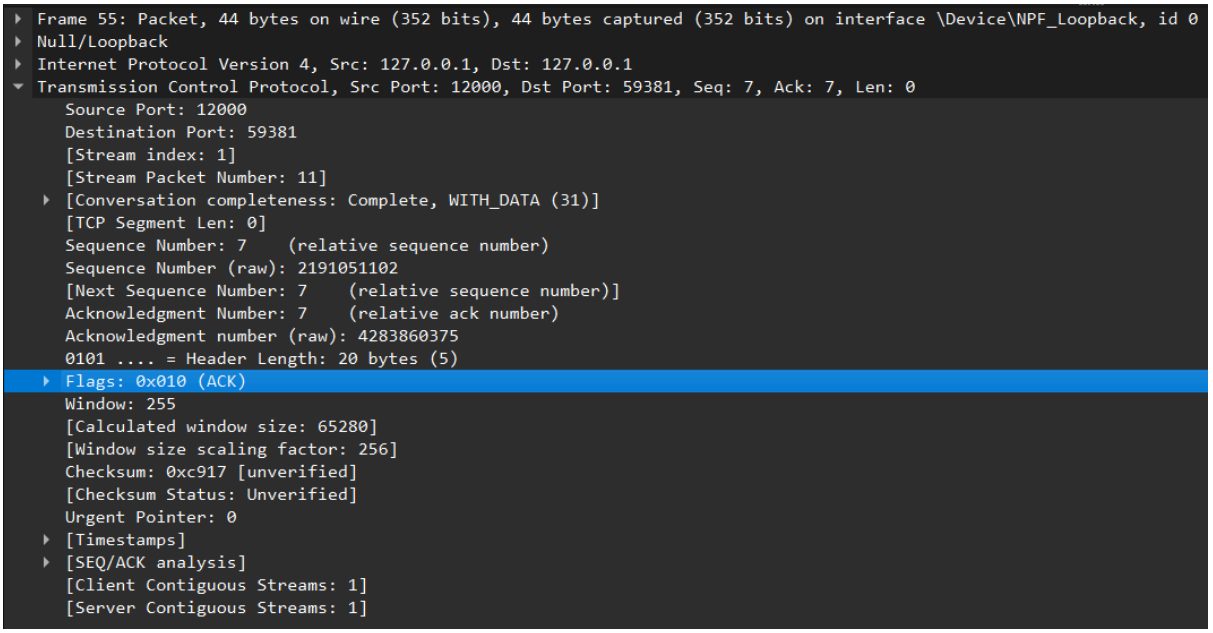
```
52 6.567809200   127.0.0.1   127.0.0.1   TCP   44 12000 → 59381 [FIN, ACK] Seq=6 Ack=6 Win=65280 Len=0
53 6.567838700   127.0.0.1   127.0.0.1   TCP   44 59381 → 12000 [ACK] Seq=6 Ack=7 Win=65280 Len=0
54 6.568344800   127.0.0.1   127.0.0.1   TCP   44 59381 → 12000 [FIN, ACK] Seq=6 Ack=7 Win=65280 Len=0
55 6.568396200   127.0.0.1   127.0.0.1   TCP   44 12000 → 59381 [ACK] Seq=7 Ack=7 Win=65280 Len=0
```

Nº	Sentido	Flags	Observação
52	servidor → cliente (12000 → 59381)	**FIN, ACK**	servidor fecha seu sentido
53	cliente → servidor (59381 → 12000)	**ACK**	confirma o FIN do servidor
54	cliente → servidor	**FIN, ACK**	cliente fecha seu sentido
55	servidor → cliente	**ACK**	confirma o FIN do cliente




Frame 53: Packet, 44 bytes on wire (352 bits), 44 bytes captured (352 bits) on interface \Device\NPF_Loopback, id 0
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 59381, Dst Port: 12000, Seq: 6, Ack: 7, Len: 0
Source Port: 59381
Destination Port: 12000
[Stream index: 1]
[Stream Packet Number: 9]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 0]
Sequence Number: 6 (relative sequence number)
Sequence Number (raw): 4283860374
[Next Sequence Number: 6 (relative sequence number)]
Acknowledgment Number: 7 (relative ack number)
Acknowledgment number (raw): 2191051102
0101 .... = Header Length: 20 bytes (5)
Flags: 0x010 (ACK)
Window: 255
[Calculated window size: 65280]
[Window size scaling factor: 256]
Checksum: 0xc918 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamps]
[SEQ/ACK analysis]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]


Frame 54: Packet, 44 bytes on wire (352 bits), 44 bytes captured (352 bits) on interface \Device\NPF_Loopback, id 0
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 59381, Dst Port: 12000, Seq: 6, Ack: 7, Len: 0
Source Port: 59381
Destination Port: 12000
[Stream index: 1]
[Stream Packet Number: 10]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 0]
Sequence Number: 6 (relative sequence number)
Sequence Number (raw): 4283860374
[Next Sequence Number: 7 (relative sequence number)]
Acknowledgment Number: 7 (relative ack number)
Acknowledgment number (raw): 2191051102
0101 .... = Header Length: 20 bytes (5)
Flags: 0x011 (FIN, ACK)
Window: 255
[Calculated window size: 65280]
[Window size scaling factor: 256]
Checksum: 0xc917 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamps]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]



b) Identifique os campos Sequence Number e Acknowledgment Number desses segmentos.

R:

Nº	Flags	Seq	Ack	Len
52	FIN, ACK	6	6	0
53	ACK	6	7	0
54	FIN, ACK	6	7	0
55	ACK	7	7	0

Pacote 52: o servidor já enviou 5 bytes (a resposta em maiúsculas), então o próximo número dele é 6. O FIN ocupa o número 6. O Ack=6 confirma os 5 bytes que recebeu do cliente.
Pacote 53: o cliente responde **Ack=7**, ou seja, "recebi tudo até o 6, inclusive o FIN, e espero o 7". O Seq=6 é o próximo byte do cliente, que ainda não enviou o seu FIN.
Pacote 54: o cliente envia o próprio FIN, com Seq=6 (o FIN ocupa o número 6 do sentido cliente para o servidor). O Ack=7 repete a confirmação do FIN do servidor.
Pacote 55: o servidor confirma com **Ack=7**, e seu Seq=7 vem do fato de o FIN do pacote 52 ter consumido o número 6.

Como no SYN, o FIN consome um número de sequência, mesmo sem carregar dados. Por isso, cada ACK de um FIN vale o Seq do FIN mais 1.

c) Explique a função das flags FIN e ACK e descreva como cliente e servidor encerram a comunicação.

R:

- FIN: indica que o emissor não tem mais dados a enviar. Fecha um sentido da conexão, e o outro sentido pode continuar ativo (*half-close*).
- ACK: confirma o recebimento, inclusive do próprio FIN. Fica ligado em praticamente todos os segmentos depois do handshake.

Como a conexão é full-duplex, cada sentido é fechado de forma independente, com um FIN e o ACK correspondente. Por isso são quatro segmentos:

1. O servidor, ao chamar `close()` depois de enviar a resposta, manda o FIN (52).
2. O cliente confirma (53). Nesse momento o sentido servidor -> cliente está encerrado, mas o cliente ainda poderia enviar dados.
3. O cliente, ao fechar o seu socket, manda o próprio FIN (54).
4. O servidor confirma (55) e a conexão termina nos dois sentidos.

Estados correspondentes:

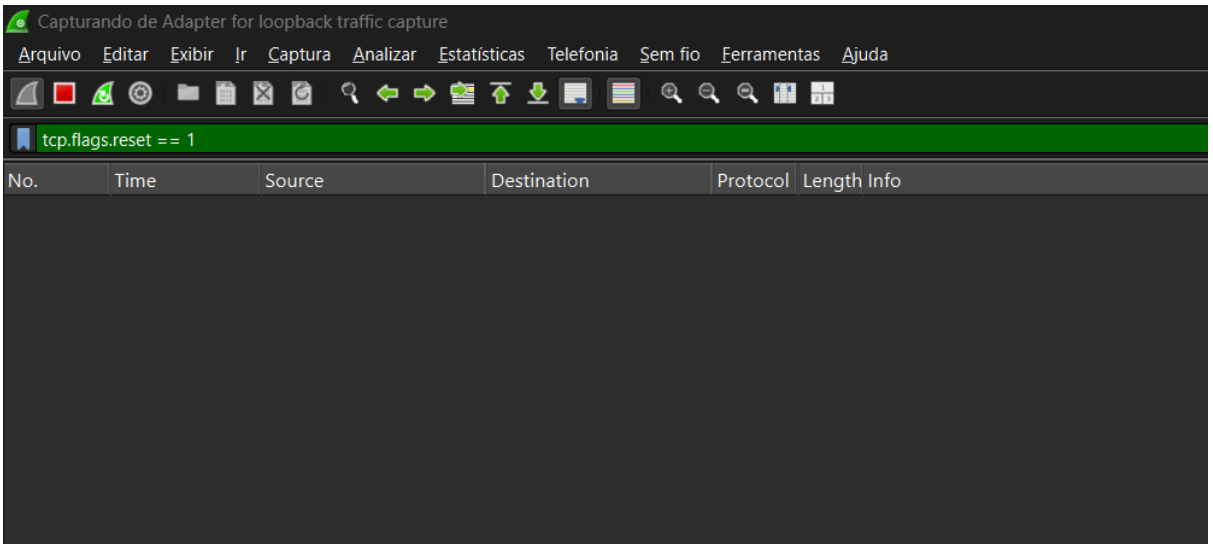
- Servidor (fechamento ativo): `ESTABLISHED` -> `FIN_WAIT_1` -> `FIN_WAIT_2` -> `TIME_WAIT` -> `CLOSED`.
- Cliente (fechamento passivo): `ESTABLISHED` -> `CLOSE_WAIT` -> `LAST_ACK` -> `CLOSED`.

O lado que fecha primeiro fica em `TIME_WAIT` por 2×MSL. Isso garante que o último ACK chegue ao outro lado e que segmentos atrasados da conexão antiga não sejam confundidos com os de uma nova conexão entre as mesmas portas.

Os pacotes 53 e 54 aparecem separados porque o cliente primeiro confirmou o FIN e só depois fechou o próprio socket. Em outras execuções, os dois podem vir juntos num único FIN, ACK, o que resulta em três segmentos.

d) Explique a diferença entre as flags FIN e RST.

R:



O filtro tcp.flags.reset == 1 não retornou nenhum pacote, ou seja, não houve RST; o encerramento foi feito apenas com FIN.

FIN:

Tipo de encerramento: ordenado (normal).

Dados pendentes: são entregues antes do fim.

Número de sequência: consome 1.

Confirmação: exige ACK.

Fechamento: um sentido por vez (half-close).

Estado final: passa por TIME_WAIT.

Valor das flags: 0x011 (FIN, ACK).

RST:

Tipo de encerramento: abrupto (aborto).

Dados pendentes: são descartados.

Número de sequência: não consome.

Confirmação: não é confirmado.

Fechamento: os dois sentidos de uma vez.

Estado final: a conexão é encerrada de imediato.

Valor das flags: 0x004 (RST) ou 0x014 (RST, ACK).

O FIN é o fechamento educado: avisa "terminei de enviar" e espera a confirmação. O RST é um sinal de erro: diz "esta conexão não é válida, pare agora". Ele costuma ser enviado quando:

Alguém tenta conectar na porta onde não há servidor escutando.

O programa termina de forma anormal ou fecha o socket com dados ainda não lidos.

Chega um segmento que não pertence a nenhuma conexão conhecida.

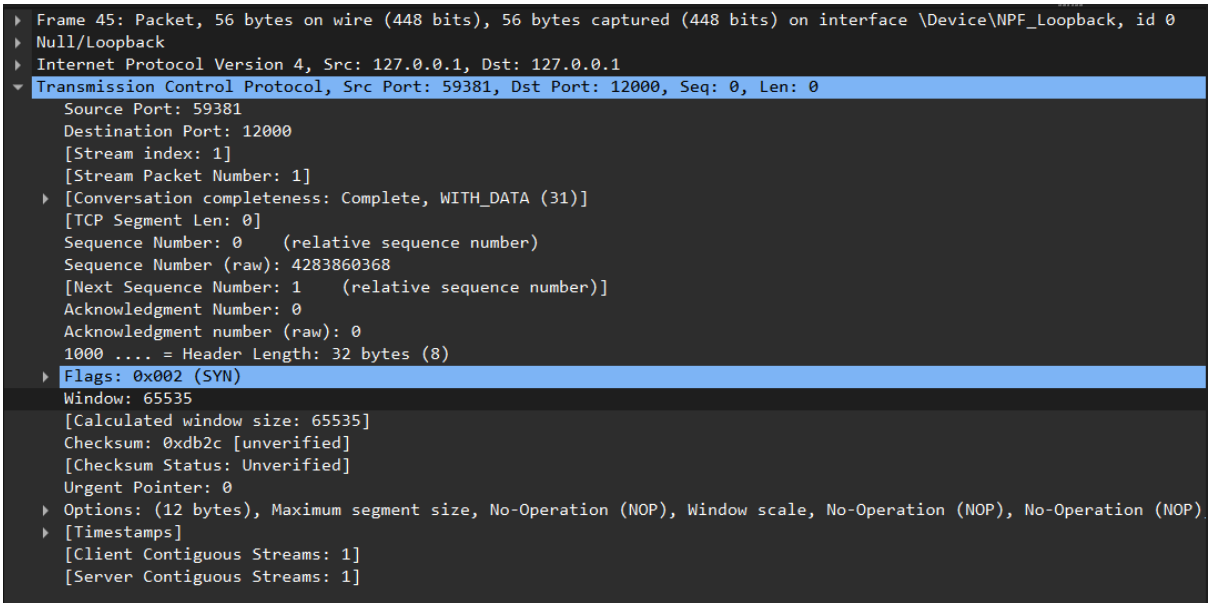
---

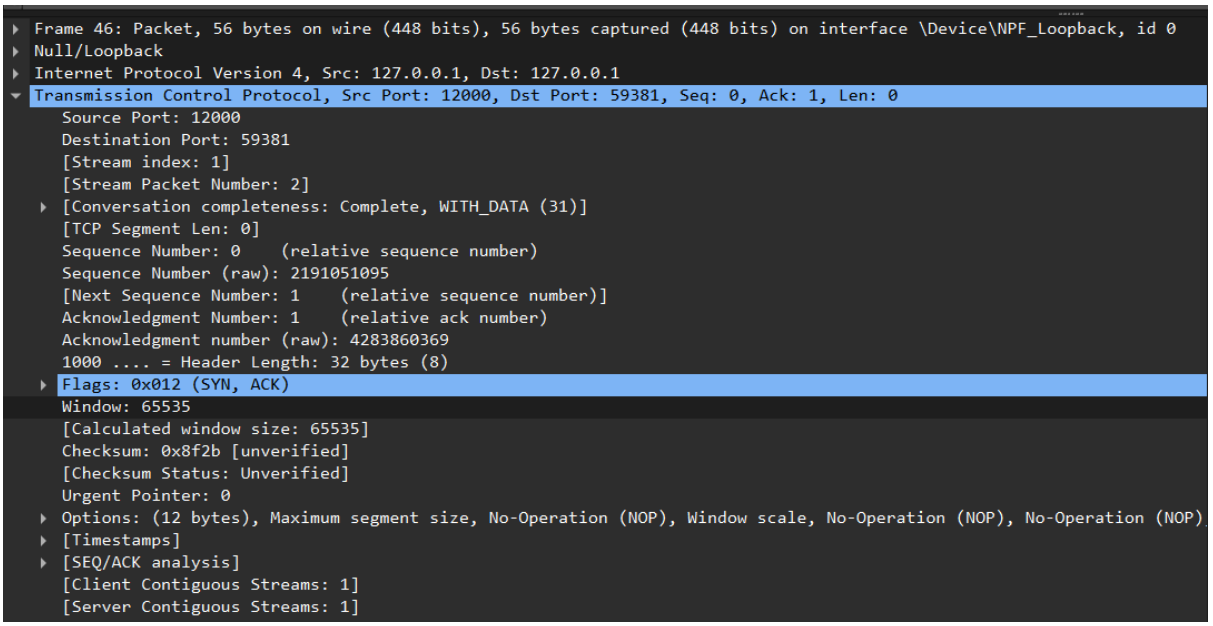
4. Controle de fluxo TCP:

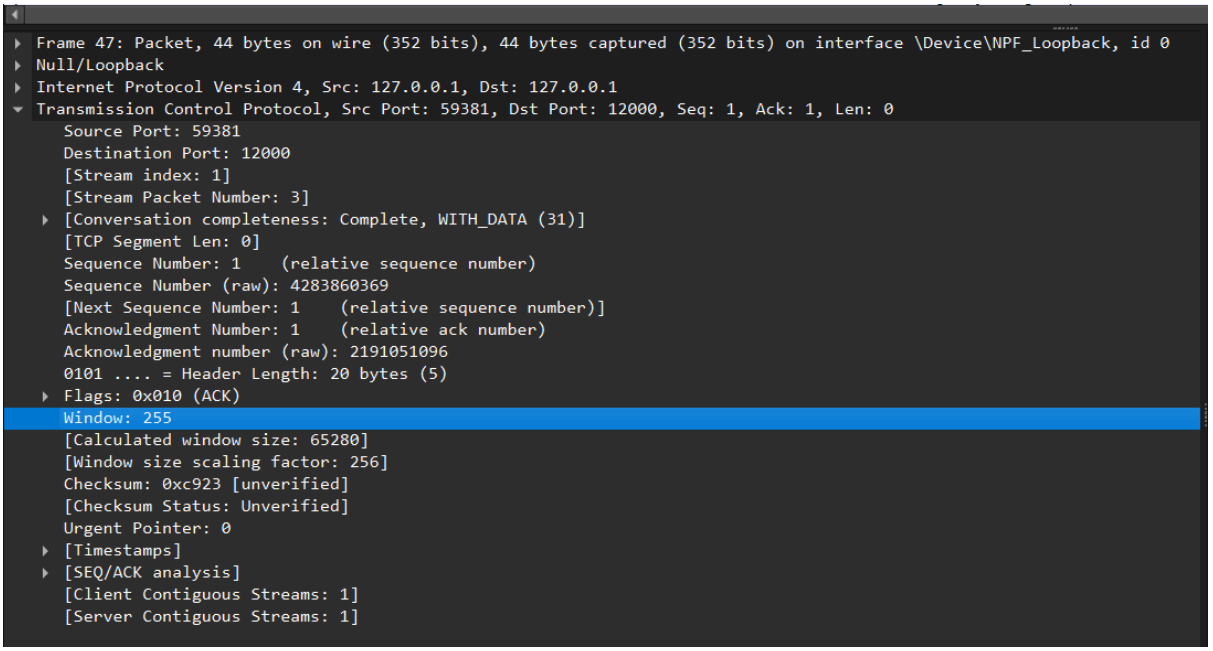
Analise os segmentos TCP trocados durante a comunicação entre o cliente e o servidor.

a) Identifique o campo Window Size (janela de recepção anunciada) nos segmentos TCP.

R:


Frame 45: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on interface \Device\NPF_Loopback, id 0
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 59381, Dst Port: 12000, Seq: 0, Len: 0
Source Port: 59381
Destination Port: 12000
[Stream index: 1]
[Stream Packet Number: 1]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 4283860368
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment number (raw): 0
1000 .... = Header Length: 32 bytes (8)
Flags: 0x002 (SYN)
Window: 65535
[Calculated window size: 65535]
Checksum: 0xdb2c [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP)
[Timestamps]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]


Frame 46: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on interface \Device\NPF_Loopback, id 0
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 12000, Dst Port: 59381, Seq: 0, Ack: 1, Len: 0
Source Port: 12000
Destination Port: 59381
[Stream index: 1]
[Stream Packet Number: 2]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 2191051095
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 4283860369
1000 .... = Header Length: 32 bytes (8)
Flags: 0x012 (SYN, ACK)
Window: 65535
[Calculated window size: 65535]
Checksum: 0x8f2b [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP)
[Timestamps]
[SEQ/ACK analysis]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]



b) Apresente o valor da janela anunciado pelo cliente e pelo servidor. Considere, quando aplicável, o fator de escala da janela (Window Scale).

R:

Nº	Quem anuncia	Flags	Win (Info)	Observação
45	cliente	SYN	65535	janela do SYN não é escalada; WS=256 anunciado
46	servidor	SYN, ACK	65535	janela do SYN/ACK não é escalada; WS=256 anunciado
47 a 51	cliente (47, 48, 51) e servidor (49, 50)	ACK / PSH, ACK	**65280**	janela escalada
52 a 55	servidor (52, 55) e cliente (53, 54)	FIN, ACK / ACK	**65280**	janela escalada

Fator de escala: os dois lados anunciaram WS=256 nos SYNs (deslocamento de 8 bits, ou seja, $2^8 = 256$). Depois do handshake, o valor bruto do campo é multiplicado por 256. O valor 65280 corresponde a um bruto de 255 (255 × 256 = 65280).

A escala existe porque o campo tem só 16 bits (máximo 65535). Com o fator 256, o TCP consegue anunciar janelas bem maiores, o que é importante em redes de alta velocidade.

c) Explique como o TCP utiliza a janela de recepção para controlar a quantidade de dados que um dispositivo pode enviar sem ultrapassar a capacidade de recepção do outro.

R: A janela de recepção (rwnd) informa quanto espaço livre o receptor ainda tem no buffer. Cada ACK carrega o valor atualizado. O emissor, por sua vez, só pode ter em trânsito, sem confirmação, uma quantidade de dados limitada por esse valor: bytes enviados e ainda não confirmados ≤ janela anunciada

Assim, um emissor rápido não afoga um receptor lento. Os pontos principais são:

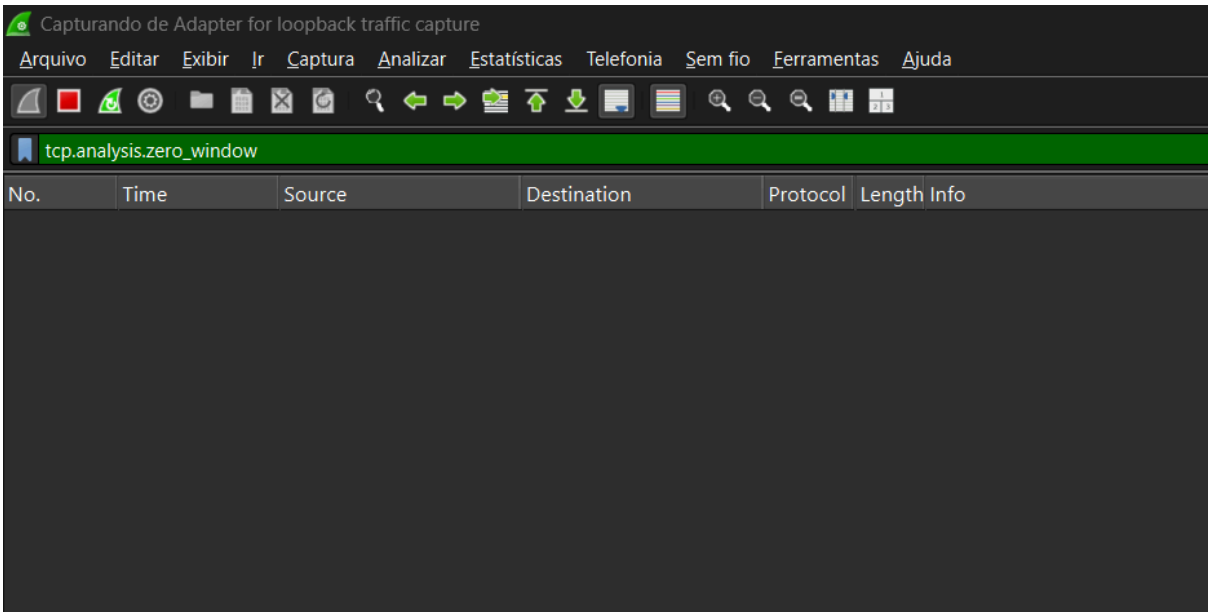
1. O receptor controla o ritmo. Ele anuncia a janela no cabeçalho de cada segmento. Quando a aplicação lê os dados do socket, o buffer esvazia e a janela cresce. Quando a aplicação lê devagar, o buffer enche e a janela diminui.
2. O emissor obedece. Ele observa o último ACK e o valor de Window recebidos e calcula quantos bytes ainda pode enviar.
3. A janela desliza. A cada novo ACK, a janela avança e libera o envio de mais dados.

Na captura, o cliente enviou 5 bytes (pacote 48) com 65280 bytes de janela disponível, então ficou muito abaixo do limite. A janela se manteve em 65280 porque a aplicação consumiu os dados imediatamente e o buffer nunca encheu. Em nenhum momento o controle de fluxo precisou frear o emissor.

Vale distinguir da janela de congestionamento (cwnd), que o emissor calcula sozinho para não sobrecarregar a rede. A taxa de envio real é limitada pelo menor dos dois valores: min(rwnd, cwnd). O campo Window do cabeçalho reflete só o limite do receptor.

d) Explique o que aconteceria se o receptor anunciasse uma janela de recepção igual a zero.

R:



O filtro tcp.analysis.zero_window não retornou nenhum pacote, pois a janela anunciada nunca chegou a zero (ficou em 65280).

Uma janela igual a zero (zero window) significa "meu buffer está cheio, não envie mais dados por enquanto". O efeito é:

1. O emissor para de enviar dados. Ele não pode mais ter bytes novos em trânsito. Segmentos sem dados (ACKs puros) e de controle (FIN, RST) ainda podem ser enviados.

2. O emissor aciona o temporizador de persistência. Se ele ficasse só esperando, poderia travar para sempre caso o aviso de janela aberta se perdesse. Por isso, ele envia periodicamente sondas de janela zero (zero window probes), segmentos pequenos que forçam o receptor a responder com a janela atual.
3. O receptor libera espaço. Quando a aplicação lê os dados do buffer, o receptor envia um ACK com a janela maior que zero (window update), e o emissor retoma a transmissão.